Center for Geospatial Analytics

NC STATE UNIVERSITY

Spatial Assessment of Carbon Storage Change Driven by Wetland Conversion

Operations Manual

Prepared by: James Jin
North Carolina State University Center for Geospatial Analytics
Master of Geospatial Information Science and Technology
In collaboration with Kelly Suttles and Adam Gold of the
Environmental Defense Fund

Environmental Defense Fund

This document is a technical overview of how to manage, edit, and use the Wetland Carbon ArcGIS Pro and Google Earth Engine tools.

Operations Requirements:

- ArcGIS Pro 3.5.2+
  - Spatial Analyst extension
  - Approximately 20 GB of storage for CONUS wide input data
- Google Earth Engine
  - Account required

Project Set Up

This project was designed to be lightweight and easy to install onto any computer and transport to any Google Earth Engine account (GEE). A folder called *wetland_carbon_analyzer* will be provided. Inside is a user guide and this operations manual. The necessary files needed for this project will be the *wetland_carbon.py* Python script, *wetland_carbon_toolbox.atbx* ArcGIS Pro toolbox file, and the *WetlandCarbonAnalyzer.js* javascript file (for GEE).

ArcGIS Pro Set Up
For the ArcGIS Pro tool, simply place the *wetland_carbon_analyzer* folder inside the desired ArcGIS Pro project folder (same level as the *.aprx* file). This will give access to the custom tool in ArcGIS Pro.

For more details about usage and data, consult the user guide for the ArcGIS Pro tool.

Google Earth Engine Set Up
The process for setting up the tool in Google Earth Engine is more involved. A GEE account and project in GEE code editor will be required to be set up beforehand. Create a new file and name it *WetlandCarbonAnalyzer* or any desired name. Copy and paste the code from the *WetlandCarbonAnalzer.js* script file into the new script in the GEE code editor. Click 'Save'.

For the necessary data for both the ArcGIS Pro tool and the GEE web application, download (CONUS extent recommended) from these links:

Download Link 2011 Wetland Soil Carbon Storage Dataset

Download Link for NLCD

Download link for Landfire Existing Vegetation Type (EVT) data

Download link for watersheds (WBD)

Migration

This section will largely focus on the migration process from the original GEE web application to another user's GEE account, involving uploading assets and making edits to the code to ensure usage of the new assets.

Uploading Assets

Data will need to be uploaded into the 'Assets' manually for each user running the script in GEE. The **2011 Wetland Soil Carbon Storage Dataset** and **2024 NLCD release** will need to be uploaded for this tool to work in GEE. Use the download links from before.

In the GEE code editor, at the left window, click on the 'Assets' tab. Click 'NEW' -> 'Image Upload' -> 'GeoTIFF'. Click 'SELECT' under 'Source files' and navigate to the **2011 Wetland Soil Carbon Storage Dataset** file. Give the asset a name and click 'UPLOAD'. Repeat this process with **NLCD release**, specifically **2024** (other release years exist within the GEE database).

In the code itself, the paths to the assets must be changed to the user's own assets. The path to assets can be found by clicking on the asset in the 'Assets' tab and copy the text under 'Image ID'.

The variable **carbon_stock_CONUS** will need to be updated with the path to the user's own **2011 Wetland Soil Carbon Storage** asset. This will be on line 13. The path for variable **nlcd_CONUS** on line 159 and 172 will also be updated but **NOT** on line 174 and 176. The NLCD paths on lines 174 and 176 call on GEE's own databases for previous NLCD releases. These should not need changing.

A comment saying "replace with…" will be above each line of code with paths that will need to be updated. Make sure to save and run again to actually use the new assets.

```
11  //uploaded 2011 CONUS full mean carbon stock data
12  //replace with path to own uploaded carbon data in assets
13  var carbon_stock_CONUS = ee.Image("projects/ee-cjjin/assets/CONUS_Full_Stock_Mean");
```

```
157  //default 2024 NLCD data
158  //replace with path to own uploaded 2024 NLCD in assets
159  var nlcd_CONUS = ee.Image("projects/ee-cjjin/assets/NLCD_LndCov_2024");
```

```
171      //replace with path to own uploaded 2024 NLCD in assets
172      nlcd_CONUS = ee.Image("projects/ee-cjjin/assets/NLCD_LndCov_2024");
```

Web Application Set Up and Code Changes

To set up as a web application, click 'Apps' at the top right of the script window. Click 'New App' and follow the prompts to set up the script as a web application. To get the link, click 'Apps' and then the newly created web application under ID. At the top of the opened window is the URL.

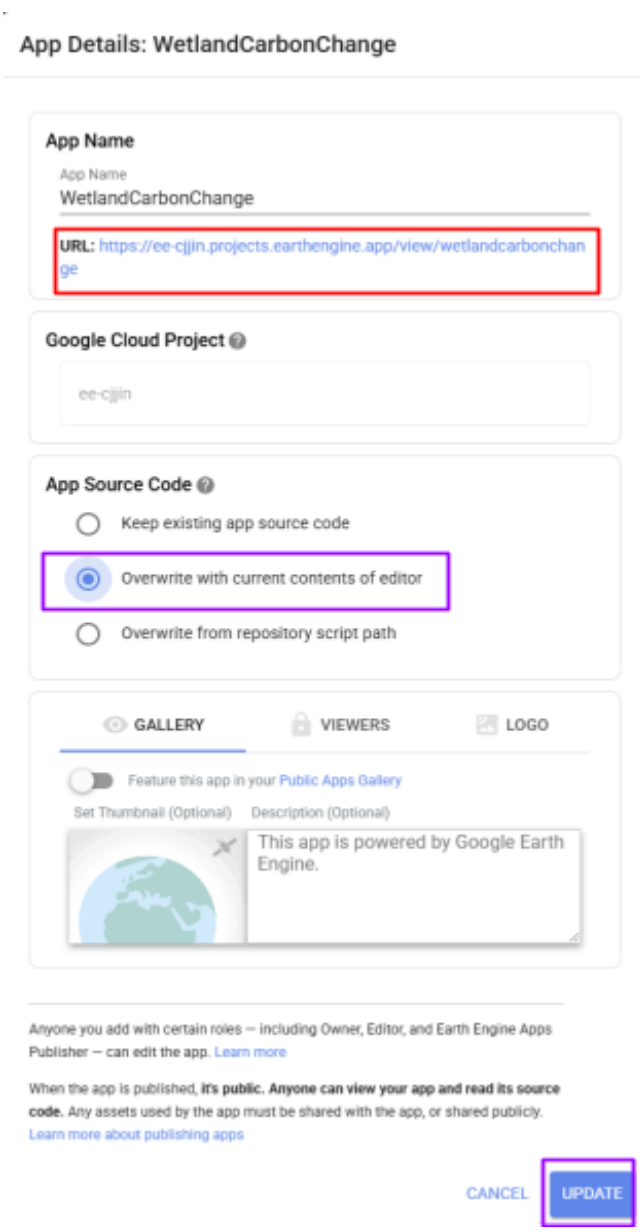
Although the web application is functional in its current state, if any changes to the code are desired, there are sections to help the user find what code to change. Sections will be defined using comments formatted as such:

```
//---------Section Name---------
```

Most code will be for the UI. If changes to the analysis are desired, scroll all the way to the bottom, the main raster calculations exist in the last function called **calculate_carbon_change()**. Make sure to click save at the top right corner of the code editor window.

To update the application after code edits are made, open the same app details window. Under 'App Source Code', select 'Overwrite with current contents of editor'. 'Overwrite with repository script path' also works if you input the proper script to update with. Then click 'UPDATE' at the bottom right.

Other properties like the logo can be changed here but these are the necessary steps to ensure that the application is accessible and updated. Make sure to update after any code changes.



Download URLs and Exporting to Google Drive

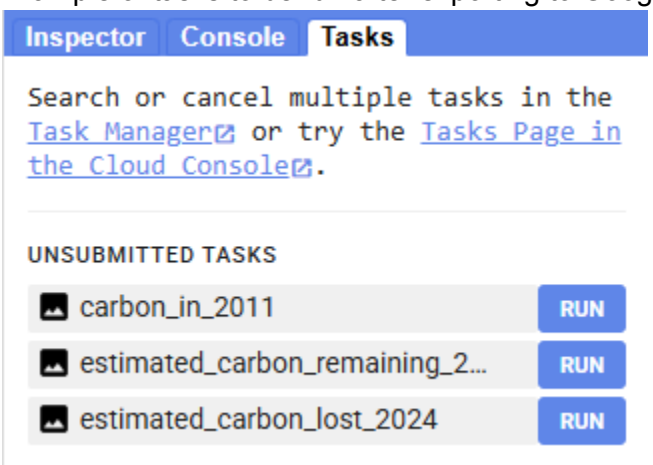
At the top of the script at line 9 is a boolean variable called **export_to_drive.** This variable controls whether or not the application generates download URLs for the calculated raster datasets or exports them to Google Drive. Set to **true** for exporting to Drive, **false** for retrieving download URLs. Since there are restrictions on download size when retrieving a download URL, there is a limit on study area size when drawing a polygon. This limit does not apply to exporting to Drive. It is still recommended to keep study area sizes reasonable (statewide).

However, exporting to Drive is not possible in the web application. It must be done when running the script in the Code Editor. When exporting to Drive is enabled, after clicking the 'Download Raster Datasets' button, new tasks will appear in the Tasks tab at the right window in the code editor. The user will then manually start each task to export to Drive.

Change this line to toggle between exporting to drive and creating download URLs.

```
//IMPORTANT: toggle true and false to switch between exporting to drive and generating download URLs.
//For web applications, only the download URLs will work.
var export_to_drive = false;
```

Example of tasks to be run after exporting to Google Drive.



Change the code shown below to alter the names and locations of the Google Drive exports. Coordinate reference system and scale can also be edited.

```
if (export_to_drive) {
  dl_panel.add(ui.Label("Check 'Tasks' in GEE Code Editor to export to Google Drive."));
  Export.image.toDrive({
    image: wetland_carbon_rast,
    description: "carbon_in_2011", //change this
    folder: "GEE_Exports", //change this
    fileNamePrefix: "c_2011", //file name
    region: study_geom,
    scale: 30,
    crs: 'EPSG:5070',
    maxPixels: 1e13
  });
  Export.image.toDrive({
    image: nlcd_carbon_rast,
    description: "estimated_carbon_remaining_" + sel_year, //change this
    folder: "GEE_Exports", //change this
    fileNamePrefix: "est_c_rem_" + sel_year, //file name
    region: study_geom,
    scale: 30,
    crs: 'EPSG:5070',
    maxPixels: 1e13
  });
  Export.image.toDrive({
    image: est_carbon_lost_rast,
    description: "estimated_carbon_lost_" + sel_year, //change this
    folder: "GEE_Exports", //change this
    fileNamePrefix: "est_c_lost_" + sel_year, //file name
    region: study_geom,
    scale: 30,
    crs: 'EPSG:5070',
    maxPixels: 1e13
  });
```

Updating

This section will detail how to make updates to the data as well as edits and modifications to the code. A brief overview of the code is provided below to help the user understand how the code works so modifications can be made.

Data Updates

The tools are designed to handle any NLCD release past 2011. For the GEE application, there will need to be updates to the code. The asset uploading process is the same, but how the code handles NLCD is unique.

As of right now, 2021 and earlier releases are held on GEE databases, with the 2021 release on a separate ImageCollection. The 2024 NLCD release was uploaded manually. When the 2024 release is added to GEE databases, that can be used instead. If/when a new NLCD is released, that will also have to be uploaded. Furthermore, the code will need updates to handle the new release.

```
157 //default 2024 NLCD data
158 //replace with path to own uploaded 2024 NLCD in assets
159 var nlcd_CONUS = ee.Image("projects/ee-cjjin/assets/NLCD_LndCov_2024");
160 //year options
161 var years = ["2013", "2016", "2019", "2021", "2024"];
162
163 var sel_year = 2024;
164 var year_select = ui.Select({
165   items: years,
166   value: "2024",
167   placeholder: "Select a year (post-2011)",
168   onChange: function(year) {
169     //use the proper NLCD data based on year
170     if (year == "2024") {
171       //replace with path to own uploaded 2024 NLCD in assets
172       nlcd_CONUS = ee.Image("projects/ee-cjjin/assets/NLCD_LndCov_2024");
173     } else if (year == "2021") {
174       nlcd_CONUS = ee.ImageCollection("USGS/NLCD_RELEASES/2021_REL/NLCD").select("landcover").first();
175     }  else {
176       var nlcd_2019 = ee.ImageCollection("USGS/NLCD_RELEASES/2019_REL/NLCD").select("landcover");
177       nlcd_CONUS = nlcd_2019.filter(ee.Filter.eq("system:index", year)).first();
178     }
179     sel_year = year;
180   }
181 });
```

The new NLCD release year will need to be added to the **years** array. In the if statements at lines 170 to 178, another conditional statement will need to be added to handle the new NCLD release.

This will likely be the only update needed for the GEE web application. As for the ArcGIS Pro custom tool, the tool is designed to take in the downloaded raster datasets as inputs. Simply download new releases for NLCD, Landfire Existing Vegetation Type (WBD), and Watershed Boundary Dataset (WBD) as needed.

Raster Calculation Workflow

The raster calculation workflow can be modified and improved in the future. To aid in this process, below is a description of the current workflow of the raster calculations. Although they are not an exact one for one, both the ArcGIS Pro and GEE code follow this general process:

1. Remaining Wetland Carbon Stock Identification
   - Wetland carbon dataset is masked to only include pixels where carbon stored is not lost. The following classes count as unchanged carbon:
     - Wetlands (90 and 95)
     - Forest (41, 42, 43)
     - Shrub/Scrub (52)
     - Grassland/Herbaceous (71)
   - Forests, shrubs, and grasslands (41-71) are included with wetlands under the assumption that wetland pixels that were converted to these land cover classes did not change in carbon stored and may alternate between wetland and other vegetation types.
   - The result is a raster that contains pixels that are unchanged in carbon stored. This is an incomplete remaining carbon stock raster.

2. Wetland Loss Detection
   - Using the same logic as in the first step but with the rest of the land cover classes produces a **carbon loss** raster with pixels of carbon stock values where wetlands were lost. The following land cover types are counted as "wetland loss":
     - Open Water (11 and 12)
     - Developed (21-24)
     - Barren (31)
     - Agriculture (81 and 82)
   - A **wetland loss-land cover** raster based on the **carbon loss** raster is created with land cover class data used instead of carbon stock values.

3. Carbon Adjustment Based on Land Cover Conversion
   - A new **carbon adjusted** raster is created based on the **carbon loss** raster. Using the land cover classes obtained from the **wetland loss-land cover** raster and the values from the **carbon loss** raster, the **carbon adjusted** raster is populated with pixels containing remaining carbon based on the user input conversion factors.
     - For this raster, the value of the pixel from the **carbon loss** raster is multiplied by (1 – conversion factor).
     - Each of the 4 general land classes (open water, developed, barren, and agriculture) have their own conversion factors defined by user inputs.
     - This **carbon adjusted** raster is combined with the previously created remaining wetlands raster from 2.1. The combined raster represents the final estimated carbon stored in soil. This is the carbon data in the "after" time frame.
   - Another **estimated carbon loss** raster is created using the same process as the **carbon adjusted** raster except the value of the pixel from the **carbon loss** raster is multiplied by the conversion factor itself.

In the GEE code, the function **calculate_carbon_change()** contains this workflow. In the Python script *wetland_carbon.py*, the method **get_carbon_rasters()** contains this workflow.

<u>Other Code in ArcGIS Pro Custom Tool</u>

This section will cover how some of the other code in the *wetland_carbon.py* script works so that it can more easily be modified. The script is designed as a class so that the code is modular and parts can be easily removed without removing whole chunks of code. The **process()** method determines what processes are run. Removing lines from this method can remove steps.

Other major functions and methods to mention are as such:
project_and_clip_rast()
For each input raster:

- Study area polygon feature class is projected to match input raster's original projection
- Data is clipped to the study area using 'Extract By Mask'
- Clipped raster is projected to **NAD83 / Conus Albers (EPSG:5070)**

get_zonal_statistics()
Calculates zonal statistics for EVT

- The 'Zonal Statistics as Table' tool is used to calculate zonal statistics for carbon stocks in the "before" time frame (original carbon soil data), using EVT as the zones.
- The same tool is used for zonal statistics for carbon stocks in the "after" time frame (estimated carbon soil data), also using EVT as the zones.
- The two resulting tables are read using Search Cursors. The vegetation type, original carbon stock sum, and estimated carbon stock sum are recorded in a Python dictionary. If a table does not contain a vegetation type that exists in the other table, 0 is recorded for that table in the dictionary.
- The Python dictionary is then written into a CSV file, with carbon values converted into metric tons of Carbon (from g C/m^2). This is handled in the **print_zonal_statistics_to_csv()** method. During this method, carbon lost is calculated by subtracting the values of the "after" time frame from the values of the "before" time frame.
- This process is done twice, once using EVT name (EVT NAME) and another for EVT subclass (EVT SBCLS). EVT name is more specific and is only recorded as in a CSV file. Along with EVT name being recorded, the numeric ID associated with the EVT name is recorded in the CSV. EVT subclass is less specific and is used to create a more visually appealing bar graph.

zonal_statistics_by_watersheds()
Calculates zonal statistics for watersheds

- The watershed input is projected to **NAD83 / Conus Albers (EPSG:5070)**. If multiple watershed feature classes were input by the user, then all were projected then merged.
- The projected (and merged) watershed feature class is clipped by the study area polygon (also projected to 5070).
- The clipped watershed feature class is then converted to a raster with the same cell size as the original wetland carbon stock data.
- The zonal statistics calculation process is the same as for Landfire EVT, mirroring the process for EVT subclass.
- This data is also printed to a CSV file, similar to Landfire EVT zonal statistics.

generate_hbar_graph()
Creates a horizontal bar graph using a dictionary of zonal statistics for carbon lost.

- A horizontal bar graph for the zonal statistics by EVT subclass is created using matplotlib.pyplot.
- The resulting bar graph is output as a PDF file.

add_to_map():
Adds the resulting raster datasets to the current map.

- ·If the tool is run in ArcGIS Pro, then three raster datasets will be added to the map, with

all previous layers (excluding basemap) removed. The raster datasets are, in order:

- Estimated lost carbon, called “carbon_lost”
    - A yellow to red color ramp is applied.
- Estimated remaining carbon, called “nlcd_carbon”
- Original carbon stored in wetland soil data, after clipping and projection, is called “carbon_stock”.

The individual steps can be altered, removed, or repurposed as the user sees fit.

Saving Updates to the ArcGIS Pro Tool

When the edits to the script file that is linked to the tool are saved, the tool in ArcGIS Pro will automatically update to run the edited script. The tool UI itself can be edited in ArcGIS Pro by right-clicking the tool in the *wetland_carbon_toolbox.atbx* toolbox, then clicking on ‘Properties’.